



Stellar Bootcamp

Stellar & Soroban Bootcamp

Prof. Josiel Maimone de Figueiredo

Maio/2026

Our Bootcamp Classes

Class 1 (05/05) — Stellar Fundamentals

Class 2 (12/05) — Custom tokens and Claimable Balances

Class 3 (19/05) — Smart Contracts with Soroban

Class 4 (26/05) — Integration and Final dApp

Whoami

“I'm just a Latin American boy with no money in the bank.
No important relatives, and from the countryside...”

Belchior

Whoami

- full professor at the Institute of Computation of the Federal University of Mato Grosso (IC-UFMT)
- Cuiabá - MT
- main focus → data engineer and blockchain (new)
- Stellar ambassador in Brazil

Class 1 – Stellar Fundamentals

Before coffee:

Your “first” step on Stellar

- Purpose and vision of the Stellar blockchain.
- Differences between Stellar and other blockchains.
- Accounts, public keys, and private keys.
- Testnet and Mainnet.
- Environment setup with Stellar.

After coffee:

Transactions and security

- Transaction structure on Stellar.
- Cryptographic signatures and account control.
- Simulation of Soroban transactions.
- Complete flow: construction, signing, simulation, and submission

Stellar Fundamentals

Your “first” step on Stellar

- Purpose and vision of the Stellar blockchain.
- Differences between Stellar and other blockchains.
- Accounts, public keys, and private keys.
- Testnet and Mainnet.
- Environment setup with Stellar.

Stellar Fundamentals

Purpose and vision of the Stellar blockchain

“The Stellar network is an **open-source blockchain** used for a variety of payment and remittance applications.

Learn about the **non-profit Stellar Development Foundation** and the fundamentals of how the network works.”

stellar.org

Purpose and vision

- **open-source blockchain**

- <https://github.com/stellar/> → organization repository
- 144 repositories → [stellar-core](#), [awesome-stellar](#), ...
- Main languages: **TypeScript**, **Rust**, **Go**, **JavaScript**, **Python** and **C++**.

- stellar-protocol

- Core Advancement Proposals (CAPs) and
- Stellar Ecosystem Proposals (SEPs).
- Protocol
- **P26** – scalability and agents (x402)
- **P25** – ZK
- **P20** – Soroban
- **P12** – SCP Stability
- ...

Purpose and vision

- **Source Code Governance**
 - **Core Advancement Proposals (CAPs)**
 - controls the stellar-core protocol
 - can change ledger and structures → affects Stellar nodes
 - **Stellar Ecosystem Proposals (SEPs)**
 - control the application layer
 - dApps and Anchor
 - adoption not mandatory

Purpose and vision

- **non-profit Stellar Development Foundation**
- **Stellar Community Fund** → [SCF Navigator](#)

Stellar Fundamentals

“Stellar powers

**borderless payments,
asset tokenization, and
DeFi solutions**

for

**builders,
institutions, and
innovators
worldwide.”**

stellar.org

Stellar Fundamentals

“Stellar powers **borderless payments**”

- Worldwide payments in 5 seconds
- Do not use other financial infrastructure (Swift, ...)
- Native DEX (Decentralized Exchange) → Anchors
- Taxes are Stellar transaction's taxes → Stroop
- Agents payment → x402 in Protocol 26

Stellar Fundamentals

“Stellar powers **assets tokenization**”

- Assets issuance are native in SCP → accessibility, simplicity and low cost
- RWA (Real World Assets) → KYC (Know Your Customer) e AML (Anti-Money Laundering)
- Scalability and privacy → thousands transactions in one contract

Stellar Fundamentals

“Stellar powers **DeFi solutions**

- Order Books → sellers and buyers connection
- Native AMM (Automated Market Makers) → Liquidity Pools
- Automated Arbitrage
- RWA Compliance

Stellar Fundamentals

“Stellar powers for **builders**”

- Rust and Soroban → easy and security programming
- Passkeys and ZK → Web3 transparency for user (UIX)
- SEP and Anchors → bank access
- x402 → AI Agents power for transactions

Stellar Fundamentals

“Stellar powers for **inovators**”

- Uses the Web3 power with Web2 simplicity
 - “dApp with Web2 UIX”

Stellar Fundamentals

Differences between Stellar and other blockchains

- Layer 1 blockchain network
 - Stellar Consensus Protocol (SCP) → FBFT – Federated Byzantine Agreement
- The blockchain architecture → block is called “ledger”
- Stable cost → < \$0,0001
- Accounts have several information → not just “ID + balance”

Stellar Fundamentals

Stellar Consensus Protocol (SCP) → 4 main properties

- **Decentralized Control** → no one dictates who is needed for consensus.
- **Low Latency** → consensus is reached in seconds.
- **Flexible Trust** → users can trust any combination of parties they deem appropriate.
- **Asymptotic Security** → security relies on digital signatures and hash families that can be scrambled against adversaries with vast computing power.

Stellar Fundamentals

Stellar Consensus Protocol (SCP) → funcionamento

- **Nomination Phase** → Produces candidate values for a slot, eventually converging to a single composite value.
- **Ballot Phase** → Uses federated voting to commit or abort ballots associated with values. If a ballot gets stuck, nodes retry with a higher ballot counter to ensure liveness.
- **Confirm Phase** → the consensus is propagated

Stellar Fundamentals

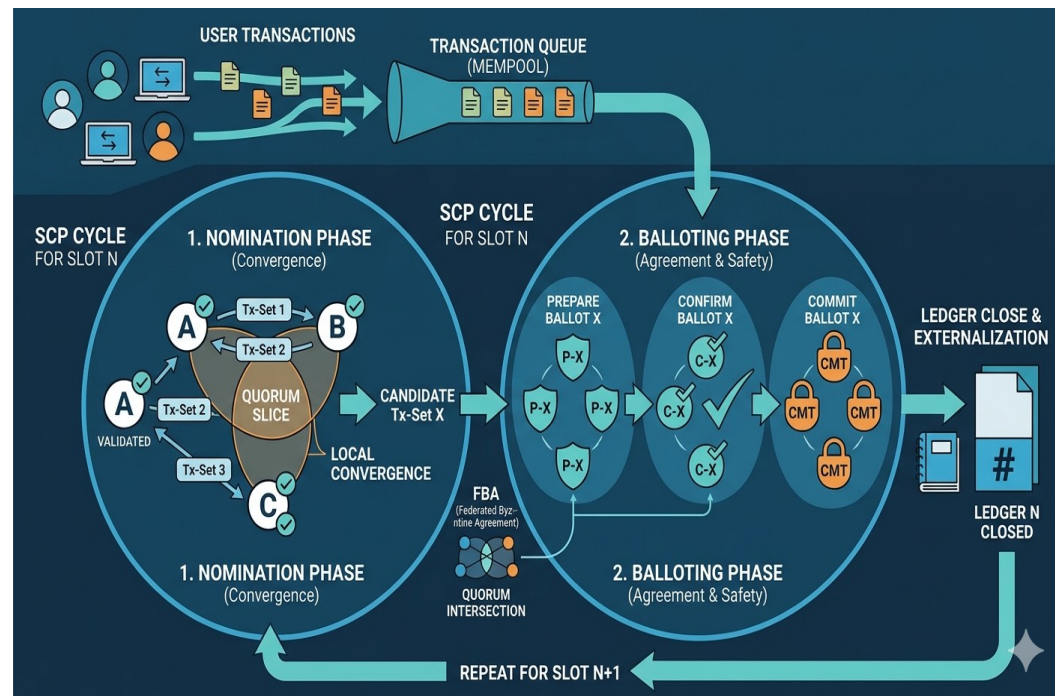
- **Nomination Phase**

- Each node indicates to their connected neighbors the transactions to be validated
- The consensus is acquired between all their confident nodes using a preconfigured quorum.
- All quorum slices propagate the consensus

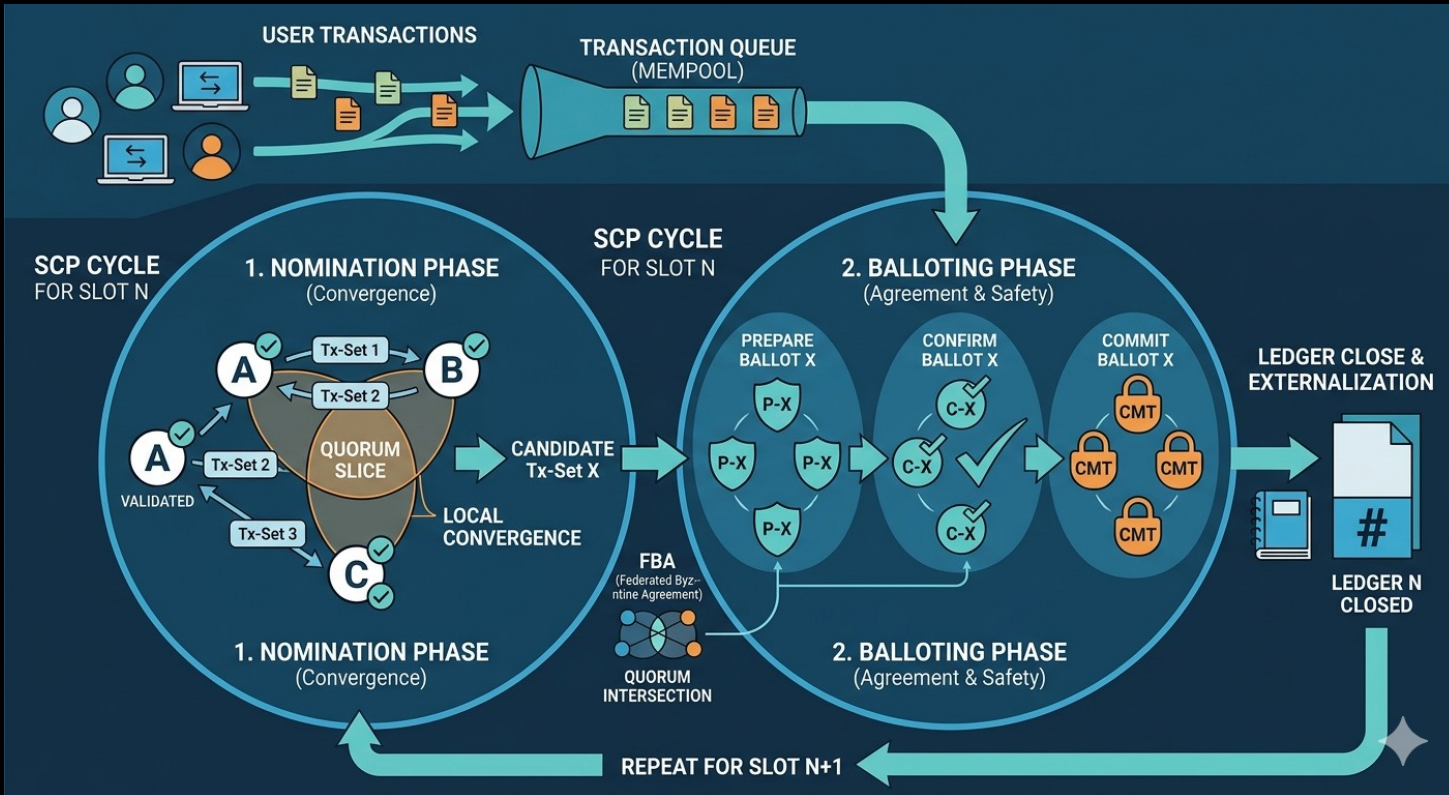
- **Ballot Phase**

- The nodes locks the chosen ballot
- Nodes votes to prepare a ballot and quorum commit it
- When node see the committed ballot, it externalize the value closing the slot (block) and the ledger.

- Simulation: <https://stellarbeat.io/>



Stellar Fundamentals



Stellar Fundamentals

But how Stellar works?

- private keys → begin with S... (56 chars)
- public keys → begin with G.... (56 chars)

Steps:

- Seed (32 bytes) → Private key (64bytes) → Str Encode → S... (56 chars)
→ Public key (32 bytes) → Str Encode → G... (56 chars)

Stellar Fundamentals

Keys and Account?

- Account will exist only after 1 XLM
- Must have a minimum balance (1 XLM + 0,5 XLM for each trustline)
- Stores all the assets
- Account stores:
 - Balance
 - Sequence number – kind of nonce for transactions
 - Trustlines – trust for assets
 - Signers – who can sign
 - Thesholds – weight for operations
 - Flags – configurations

Stellar Fundamentals

Testnet and Mainnet

- Testnet → for developers and can be reseted
 - Account creation via Friendbot
- Mainnet → production

Stellar Fundamentals

Testnet and Mainnet

- From Testnet to Mainnet
 - Change Horizon endpoint from [horizon-testnet](https://horizon-testnet.stellar.org) to horizon.stellar.org
 - Update network passphrase
 - Generate new keypairs
 - Fund accounts with real XLM
 - Update any hardcoded asset code
 - Test with small amounts
 - Consider multisig

Stellar Fundamentals

- A private Stellar Network
 - Local Network – multi node
 - Standalone – one node

Stellar Fundamentals

Environment setup with Stellar

- Two main cases:
 - Add a new node on Stellar Network
 - Connect to Stellar Network for dApp development

Stellar Fundamentals

Environment setup with Stellar (add new Node)

- Configure node types:
 - **Watchers** → for exchanges/services needing private ledger access
 - **Validators** (basic or full) → for institutions
- Set Quorum Slices → must choose nodes to trust and other existing nodes should trust on the new one
- Run stellar-core

Stellar Fundamentals

Transactions and security

- Transaction structure on Stellar.
- Cryptographic signatures and account control.
- Simulation of Soroban transactions.
- Complete flow: construction, signing, simulation, and submission.

Stellar Fundamentals

Transaction structure on Stellar.

- Composed by a set (1..100) of operations
 - **Operation** → command that modify the ledger
- **Transaction is atomic** → all operations or none
- **Envelope** → transaction + signature
 - XDR (External Data Representation) → RFC4506 binary standard
 - Optimized for network performance (small and fast)
 - Not human-readable (requires SDKs to decode)
 - Fundamental to the protocol (all transactions, ledgers, and messages use XDR)

Stellar Fundamentals

Transaction structure on Stellar.

- Preconditions
 - **Time Bounds** (min_time / max_time) → Transaction is only valid within a specific time window (UNIX timestamps)
 - **Ledger Bounds** (min_ledger / max_ledger) → Transaction is only valid within a specific ledger range
 - **Minimum Account Sequence** → Requires the source account's sequence number to be at least a certain value
 - **Minimum Sequence Age** → Time (in seconds) that must have passed since the sequence number changed
 - **Extra Signers** → Up to two additional signers that must sign the transaction

Stellar Fundamentals

Transaction structure on Stellar.

- Fields:
 - **id** (string) → Unique identifier for this transaction
 - **hash** (string) → Hex-encoded SHA-256 hash of the transaction's XDR-encoded form
 - **ledger** (number) → Sequence number of the ledger that included this transaction
 - **created_at** (ISO8601 string) → transaction timestamp
 - **source_account** (string) → Account that originates the transaction
 - **source_account_sequence** (string) → Sequence number consumed by this transaction
 - **fee_charged** (number) → Actual fee paid (100 stroops) plus variable contract costs

Stellar Fundamentals

Transaction structure on Stellar.

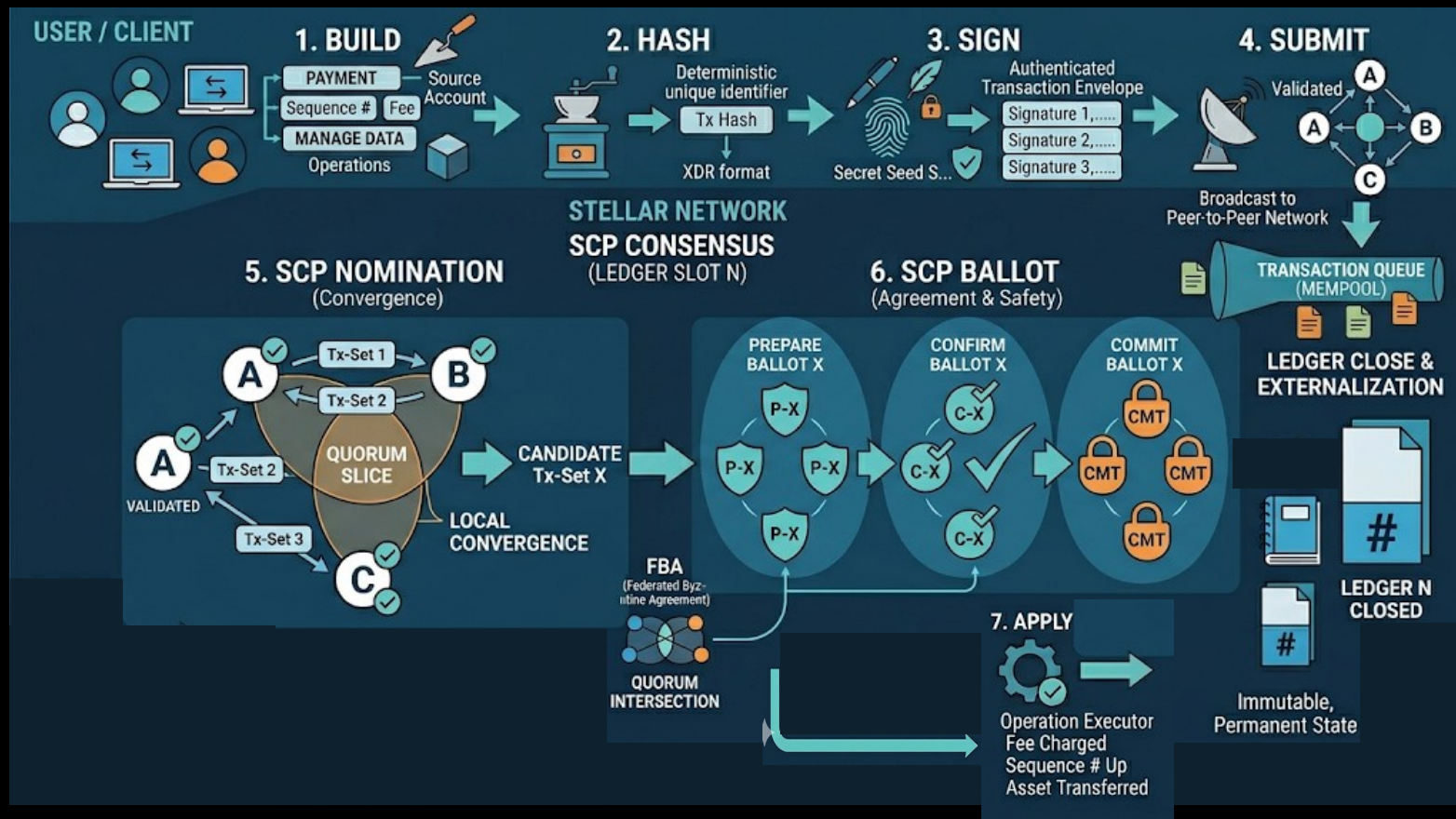
- Fields (cont.):
 - **max_fee** (number) → Maximum fee the source account was willing to pay
 - **operation_count** (number) → Number of operations in this transaction
 - **successful** (boolean) → Whether the transaction succeeded
 - **memo** (string) → Optional memo attached to the transaction
 - **memo_type** (string) → Type: MEMO_TEXT, MEMO_ID, MEMO_HASH, or MEMO_RETURN
 - **signatures** (array of strings) → Signatures used to authorize this transaction (up to 20)

Stellar Fundamentals

Transaction lifecycle (8 phases)

- 1) Build → Create operations and bundle them into a transaction (using SDK)
- 2) Hash → transaction id generated (different from Nets)
- 3) Sign → Sign the transaction hash with secret key(s), with weight and threshold applied
- 4) Submit → Send the TransactionEnvelope to a Horizon endpoint
- 5) SCP nomination
- 6) SCP ballot
- 7) Apply → Network validates, includes in a ledger, executes atomically
- 8) Ledger closed → “block chained”

Stellar Fundamentals



Stellar Fundamentals



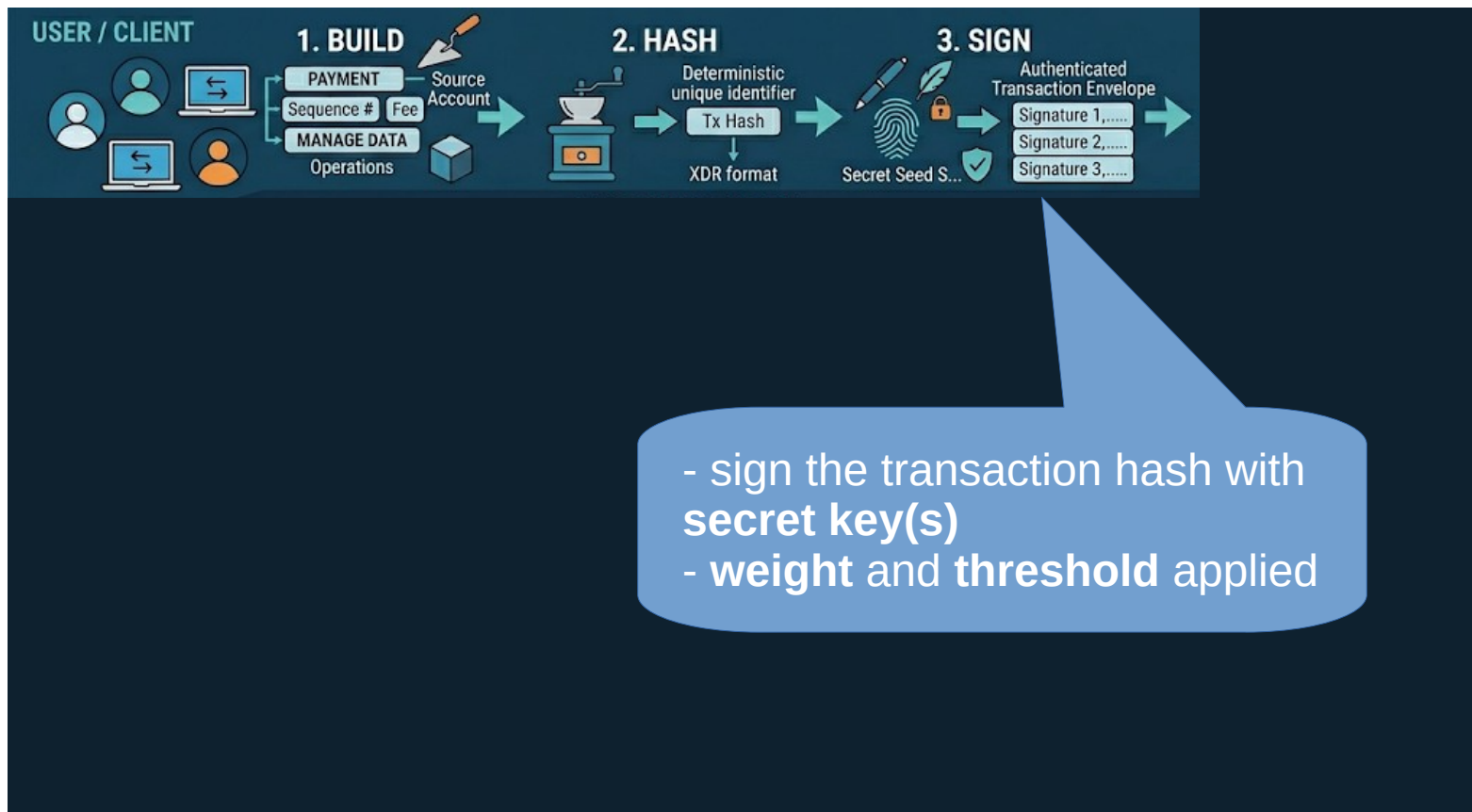
Create operations and
bundle them into a
transaction

Stellar Fundamentals

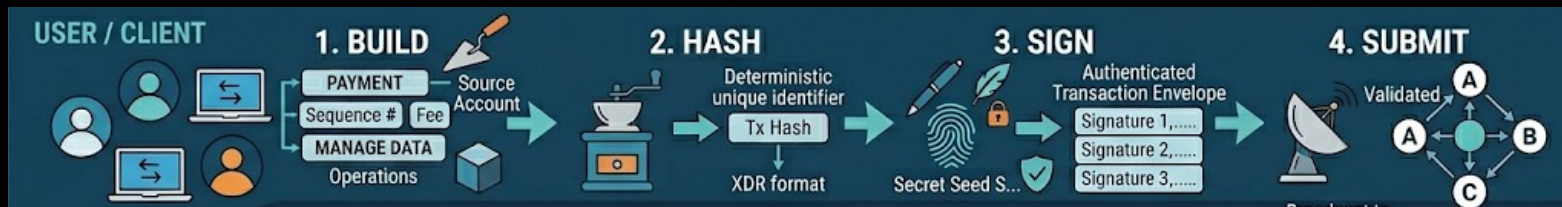


- transaction id generated
- different for each Net (Mainnet, TestNet, FutureNet)

Stellar Fundamentals

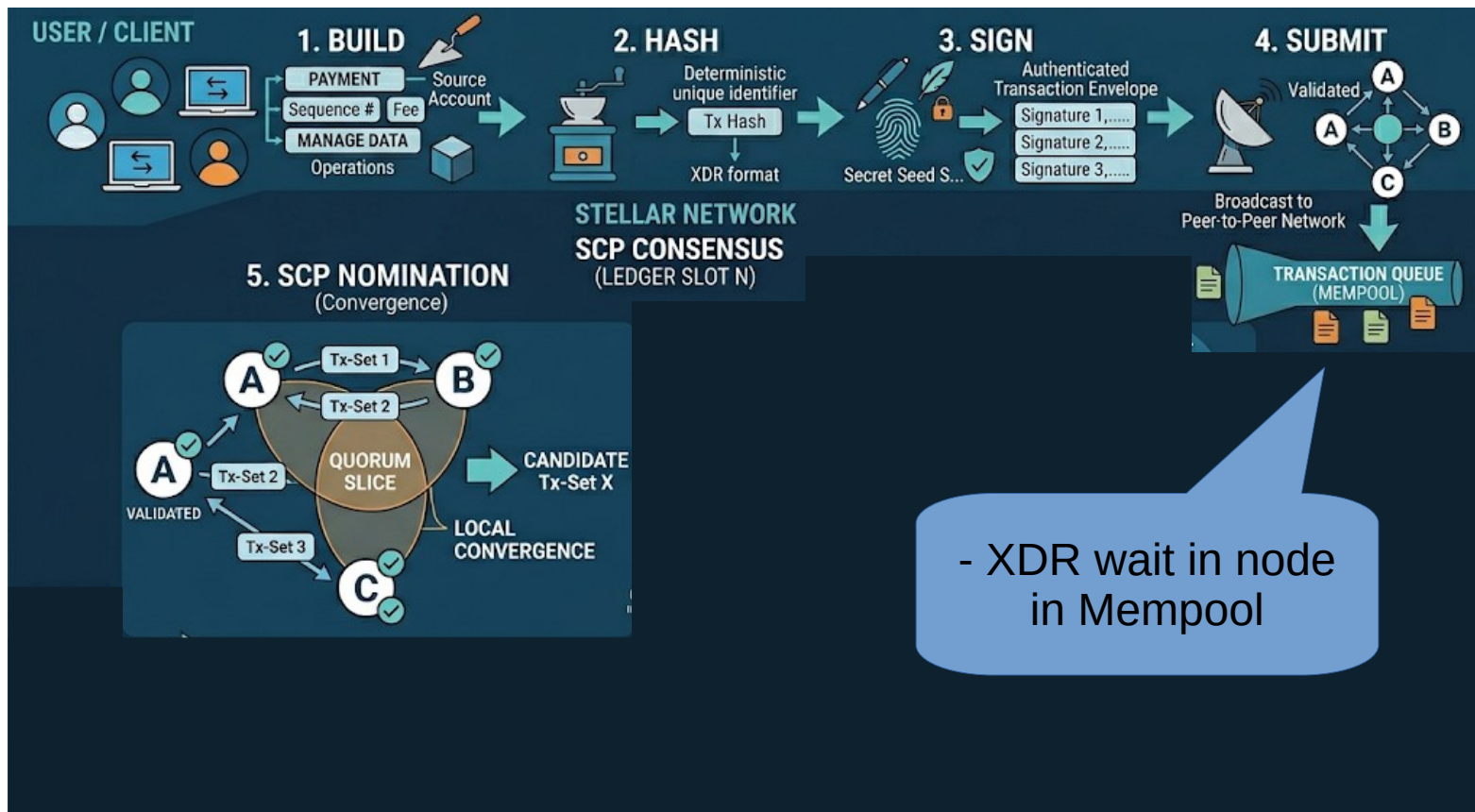


Stellar Fundamentals

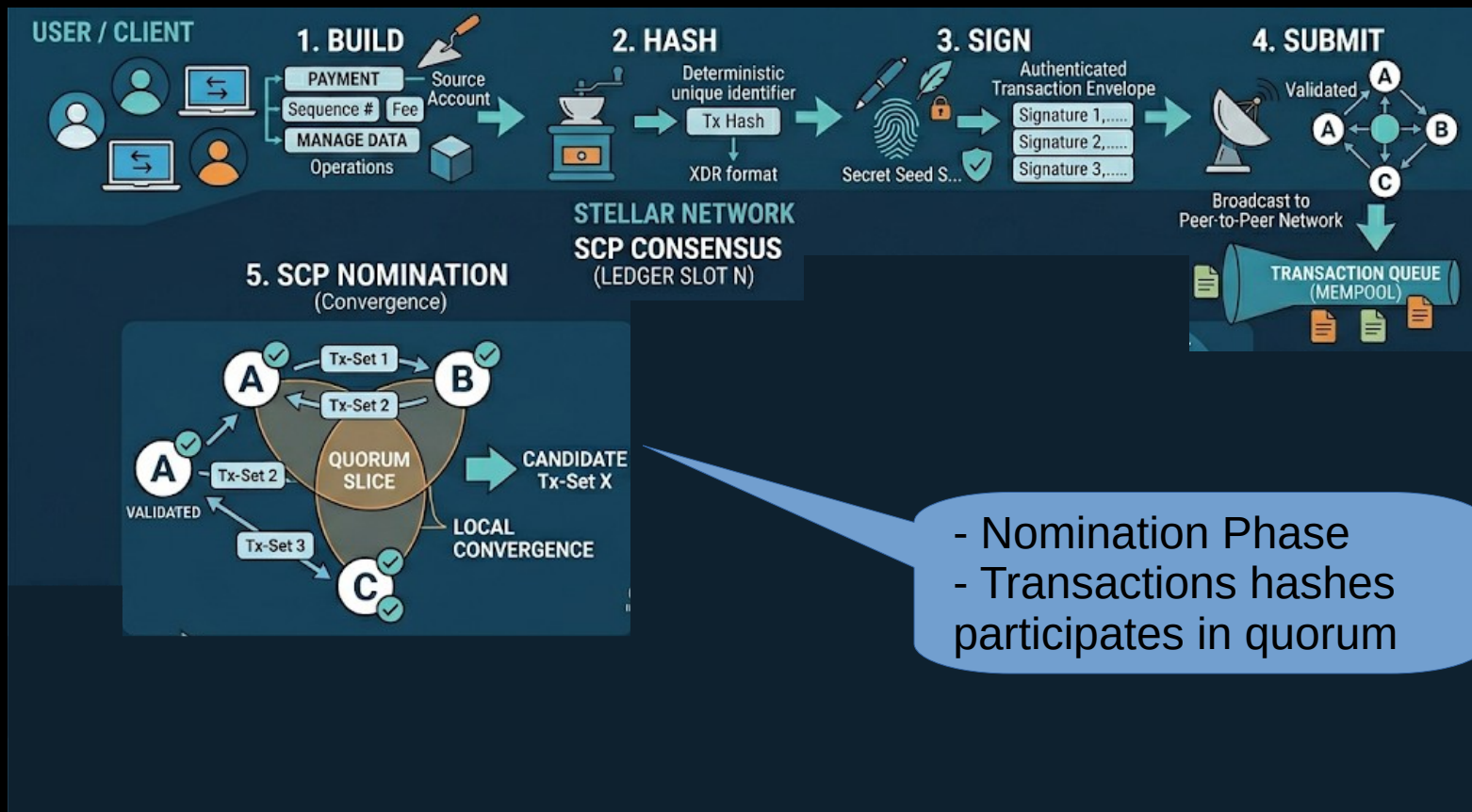


- send the Transaction Envelope to a Horizon endpoint
- if **transaction invalid** then **no fees** and **no increment of sequence**

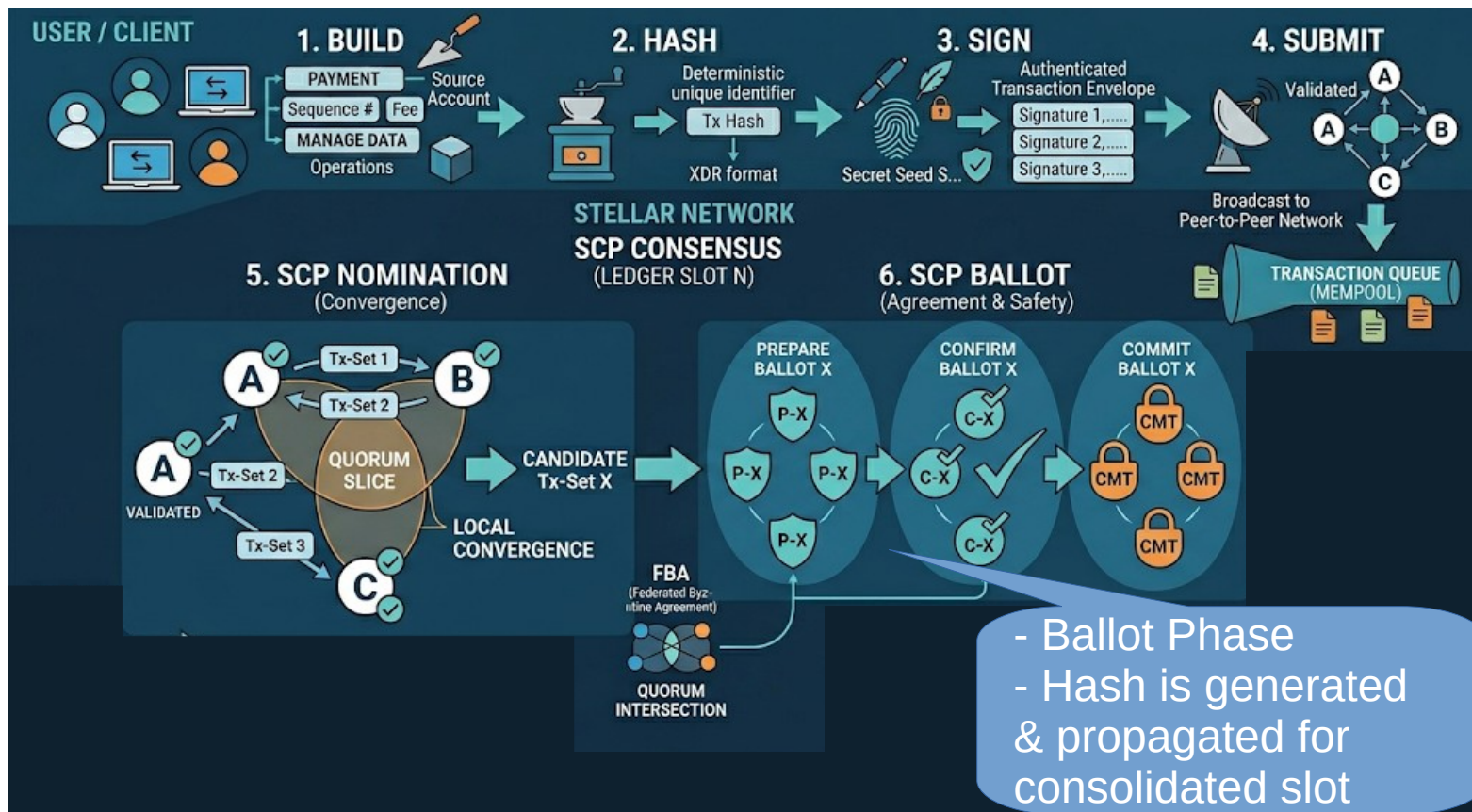
Stellar Fundamentals



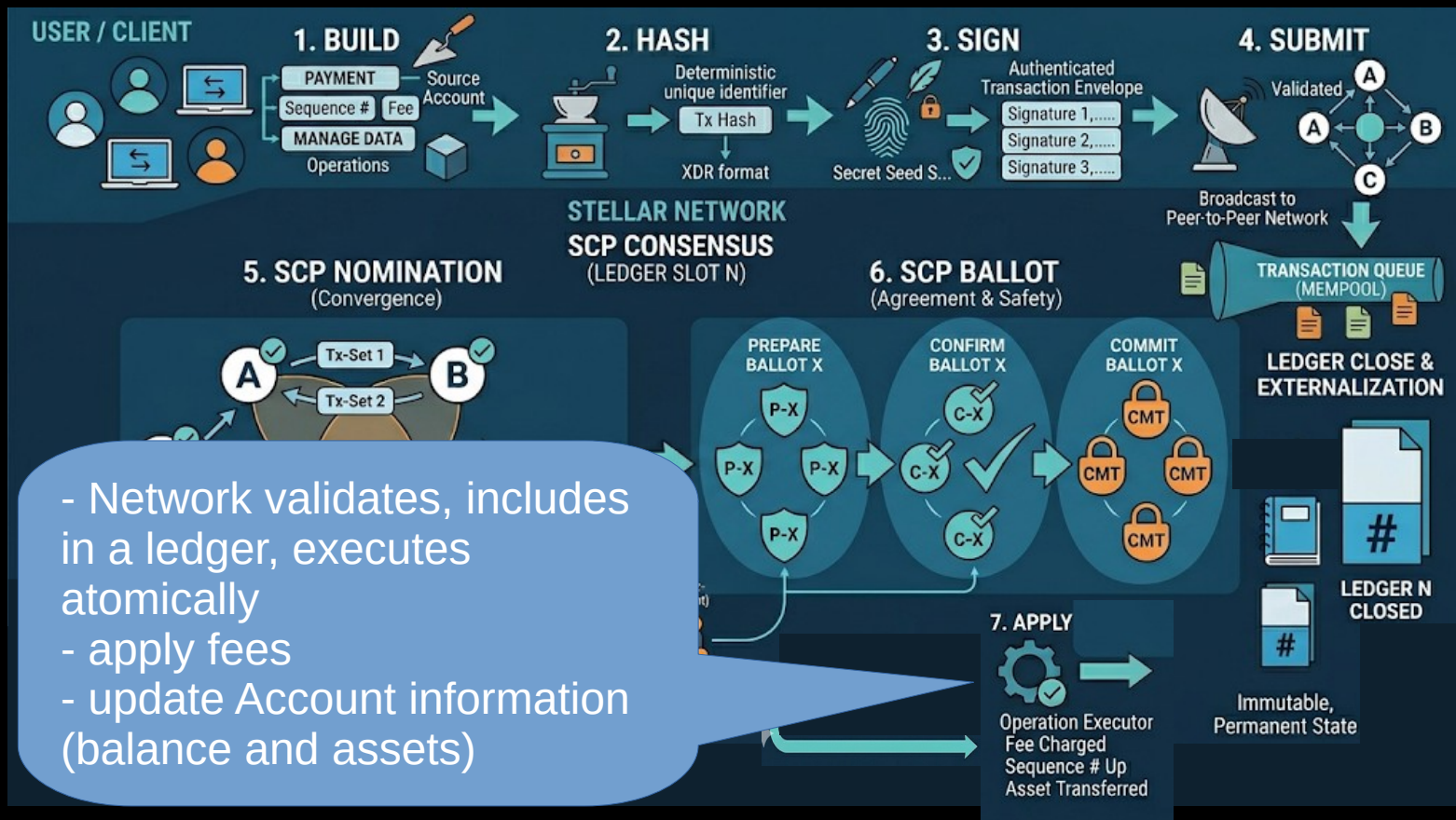
Stellar Fundamentals



Stellar Fundamentals



Stellar Fundamentals



Stellar Fundamentals



Stellar Fundamentals

Cryptographic signatures and account control.

- Stellar uses the **Ed25519** signature scheme as its standard
 - A transaction is signed using a secret key (seed) or master key
 - Default Weight =1 → if is set to zero than account is lock for the master key
 - The signature is attached to the transaction envelope, allowing the network to verify it against its public key.
 - But account control uses flexible threshold system that allows different levels of security to different types of operations.
- For performance rule → if a transaction has more signatures than are strictly required to meet the threshold, it will fail with a TX_BAD_AUTH_EXTRA error

Stellar Fundamentals

Cryptographic signatures and account control.

- Threshold Levels depend on operation type (default=0):
 - **Low** → Allow Trust, Bump Sequence, Claim Claimable Balance
 - **Medium** → Payments, Offers, Path Payments, Create Account, Manage Data, etc.
 - **High** → Set Options (changing signers/thresholds), Account Merge

Stellar Fundamentals

Cryptographic signatures and account control.

- Threshold Levels depend on operation type (default=0):
 - **Low** → Allow Trust, Bump Sequence, Claim Claimable Balance
 - **Medium** → Payments, Offers, Path Payments, Create Account, Manage Data, etc.
 - **High** → Set Options (changing signers/thresholds), Account Merge

Stellar Fundamentals

Some Questions to remember about today:

- A transaction on Stellar has the following property regarding its operations:
 - A) Each operation is executed independently, even if others fail
 - B) The entire transaction is atomic; if one operation fails, all operations fail.
 - C) Operations are executed in random order for fairness
 - D) A transaction can contain an unlimited number of operations
 - E) Smart contract transactions can contain up to 100 operations

Stellar Fundamentals

Some Questions to remember about today:

- In the lifecycle of a Stellar protocol upgrade, what is the primary difference between a CAP (Core Advancement Proposal) and a SEP (Stellar Ecosystem Proposal)?
 - A) CAPs are for smart contracts and SEPs are for payments
 - B) CAPs are written by the SDF, and SEPs are written by the community
 - C) CAPs change the core protocol and require a validator vote, while SEPs standardize application-layer patterns.
 - D) CAPs are implemented in Rust, and SEPs are implemented in JavaScript
 - E) There is no difference; they are two names for the same process

Stellar Fundamentals

Some Questions to remember about today:

- What are the three configurable threshold levels (Low, Medium, High) used for in a classic Stellar account?
 - A) To define the minimum balance required for the account
 - B) To set different levels of signature weight required for different types of operations.
 - C) To determine how fast a transaction is processed
 - D) To limit the number of transactions per day
 - E) To specify which assets the account can hold

Stellar Fundamentals

Some Questions to remember about today:

- What is the primary data structure used by Stellar's core protocol to manage ledger state, instead of a traditional Merkle Tree?
 - A) Hash Chain
 - B) Bucket List
 - C) Patricia Trie
 - D) AVL Tree
 - E) Sparse Merkle Tree



Stellar Bootcamp

Próximo encontro (12/5):

Custom tokens and Claimable Balances